

LSTM — Long Short-Term Memory

(Part 1: The “What?”)

➤ Goal:

Fix the problems of regular RNNs, mainly **vanishing gradients** and **short-term memory**, so that the model can *remember important information* for longer sequences.

1. The RNN Limitation

An RNN passes information from one step to the next through its hidden state:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1})$$

But:

- Gradients **vanish** as we go back in time.
- It **forgets** information from earlier steps.
- So it can't connect long-term relationships in the data.

Example:

"The cat that chased the mouse was **hungry**."

A normal RNN might forget that “cat” is the subject by the time it predicts “hungry.”

We need a way to **remember key information longer** and that's what LSTM does.

2. LSTM Core Idea

LSTM introduces a “**memory cell**”, a component that acts like a **long-term memory storage**.

It can **keep important info** for many time steps and **decide when to update or forget it**.

So unlike RNNs (which just pass everything forward),

LSTMs *control the flow of information* using **gates**.

➤ Structure of an LSTM Unit

Each LSTM cell has:

1. **Cell state (C_t)** → the “long-term memory”
 2. **Hidden state (h_t)** → the “short-term memory” (used for output)
 3. **Gates** → control how info flows in and out
-

3. The Three Gates (and One Output)

Gates are **tiny neural networks** with sigmoid (σ) activations, values between **0 and 1**, that control how much info passes through.

Gate	Purpose	Symbol	Function
Forget Gate	Decides what to remove from memory	(f_t)	Forget old info
Input Gate	Decides what new info to add	(i_t)	Add useful new info
Output Gate	Decides what to output	(o_t)	Control what goes out

4. Step-by-Step Flow of an LSTM

Let's walk through one time step in a simple way.

► Step 1: Forget Gate

Decide what to throw away from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- If $f_t = 0$: forget everything
- If $f_t = 1$: keep everything

Example: Forget irrelevant past info like punctuation or filler words.

► Step 2: Input Gate

Decide what new information to store.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- i_t : how much to allow in
- $\tilde{C}_t$: candidate values to be added

► Step 3: Update Cell State

Combine old memory (after forgetting) with new info.

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

So the cell state **retains what's useful** and **adds new knowledge**.

► Step 4: Output Gate

Decide what part of the cell state should become output.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

The hidden state h_t is then used:

- as output for the current step
- and passed to the next step

5. Intuitive Summary

- **Forget Gate:** "Should I delete this memory?"
- **Input Gate:** "Should I add this new info?"
- **Cell State:** "The running memory of what's important."
- **Output Gate:** "What should I show/output this time?"

Analogy:

Think of LSTM like your brain managing memory:

- You forget useless details,
- You remember what's relevant,
- You update your understanding,
- You express only the useful part.

6. Why LSTM Solves RNN's Problems

Problem (RNN)	LSTM Solution
Vanishing gradients	Cell state allows constant gradient flow over time
Long-term dependencies	Memory cell stores info across many steps
Forgetting important context	Controlled by input/forget gates
Slow learning	Stable gradients = faster convergence

Because of the **cell state**, gradients can flow without decaying — so the network “remembers” patterns that occurred much earlier.

7. Visual Summary

Imagine a **conveyor belt (cell state)**:

- Information flows down the belt mostly unchanged.
- Three gates stand by:
 - **Forget Gate:** removes irrelevant items.
 - **Input Gate:** adds new useful items.
 - **Output Gate:** chooses what to reveal at the current step.

This simple mechanism makes LSTM *much better at remembering* compared to vanilla RNNs.

8. Mathematical Overview (for Reference)

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

9. Summary — LSTM in One Look

Concept	Meaning
LSTM	Long Short-Term Memory, an improved RNN
Key Idea	Keep important info longer, forget unimportant
Cell State (C_t)	Long-term memory
Hidden State (h_t)	Short-term output
Gates	Small neural networks that control info flow
Advantage	Solves vanishing gradient and remembers longer sequences
Inspiration	Human memory (store → forget → recall)

➤ Final Takeaway

LSTM is the memory-enhanced version of RNN.

It uses gates to control what to keep, what to forget, and what to output allowing it to learn patterns across long sequences.
